

结构化程序设计

北京邮电大学计算机学院

软件与理论中心 张艳梅



伪代码表示的算法结构

伪代码是介于过程步骤的自然语言算法描述，和用实际编程语言描述过程之间的中间体。伪代码的基本结构类似于编程语言的控制结构，但比编程语言规范更自由，允许自然语言的步骤描述。

```
procedure max( $a_1, a_2, \dots, a_n$ : integers)  
    max :=  $a_1$   
    for  $i := 2$  to  $n$   
        if max <  $a_i$  then max :=  $a_i$   
    return max{max is the largest element}
```

大括号{}在伪代码中表示注释，而用缩进表示嵌套的关系。

- 首先定义算法的名称（功能），列出输入数据及对应的数据类型。
- 赋值运算语句，可以包含任何良定义的运算，即使它复杂到用许多编程语句才能实现。例：
 $x :=$ 列表L中的最大整数。
- 控制结构使用：

```
if 条件 then 语句 1  
else 语句 2
```

```
while 条件  
    语句
```

```
for 变量 := 初值 to 终值  
    语句
```

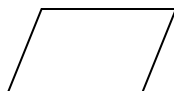
- 最后使用return语句说明何时输出什么内容。

程序的结构描述—流程图

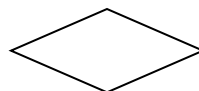
- 流程图的基本元素（ANSI规定）



起止框



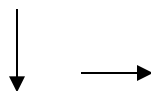
读入/出框



判断框



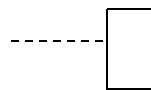
处理框



流程线



连接点



注释框

求10! 流程图

使用当型循环结构描述

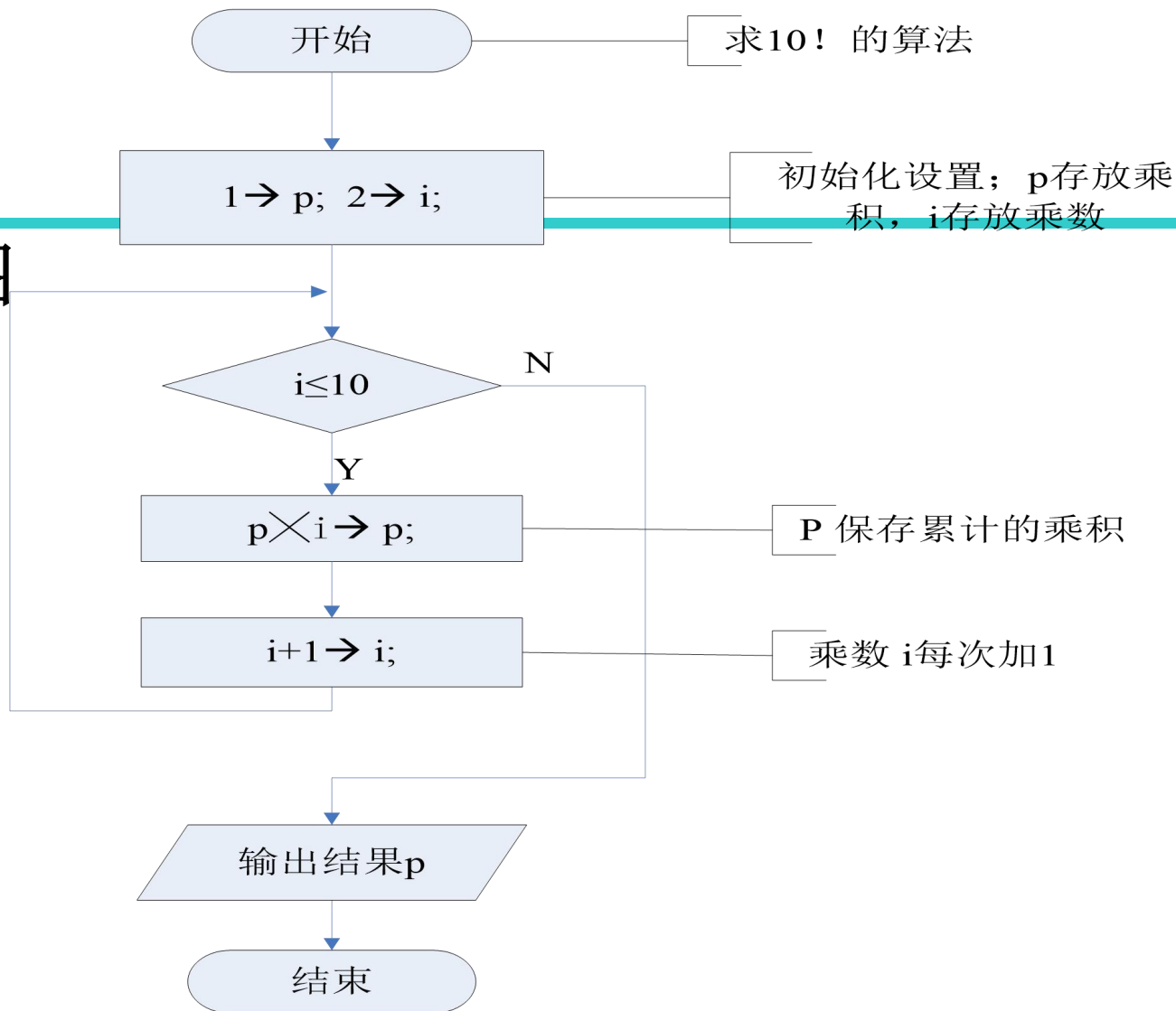
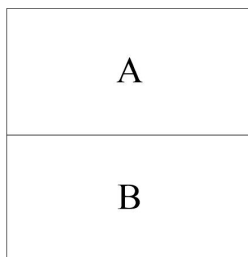


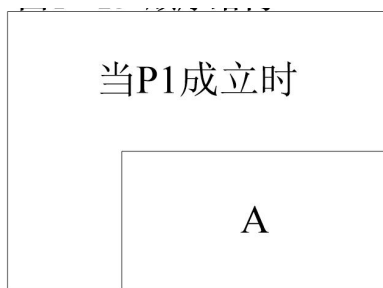
图4.10 求10! 并输出结果的算法流程图

程序的结构描述—N-S流程图

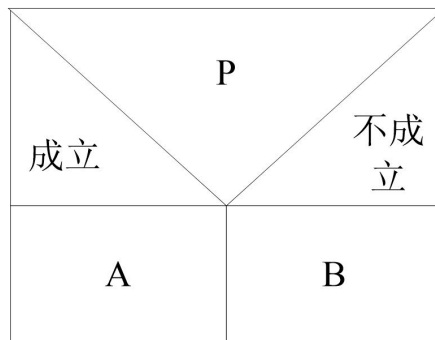
- 基本结构



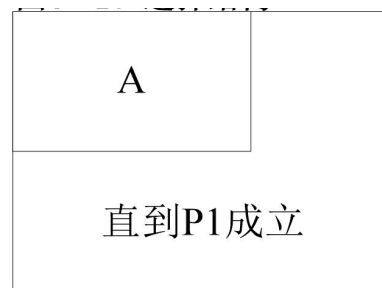
顺序结构



当型循环结构



选择结构



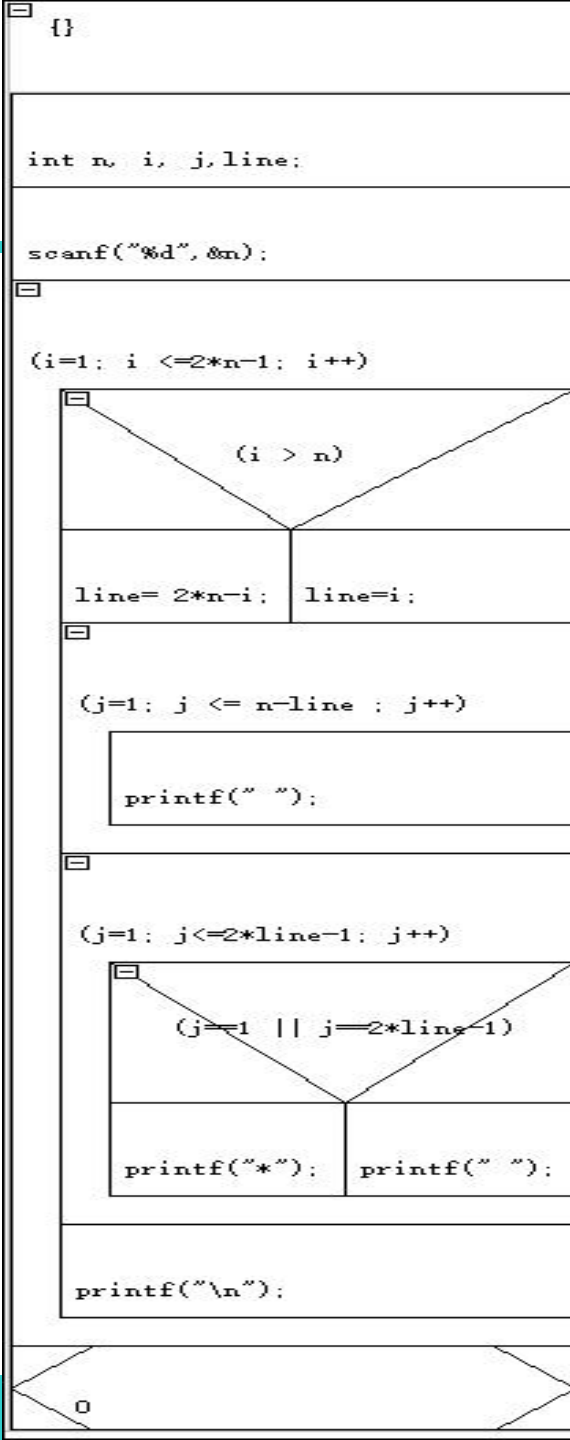
直到型循环结构

p、p1表示的是判断条件；A、B框是操作；

注意：A、B框可以是一个简单的操作（如读入数据或打印打印等），也可以是三种基本结构之一



```
#include <stdio.h>
int main()
{
    int n, i, j, line;
    scanf("%d", &n);
    for (i=1; i <=2*n-1; i++){
        if (i > n) line= 2*n-i; else line=i;
        for (j=1; j <= n-line ; j++) printf(" ");
        for (j=1; j<=2*line-1; j++)
            if (j==1 || j==2*line-1) printf("*");
            else printf(" ");
        printf("\n");
    } //end for(i)
    return 0;
} //end main()
```



结构化程序设计方法的起源

```
#include <stdio.h>
#include <stdlib.h>
main()
{
    int count=1;

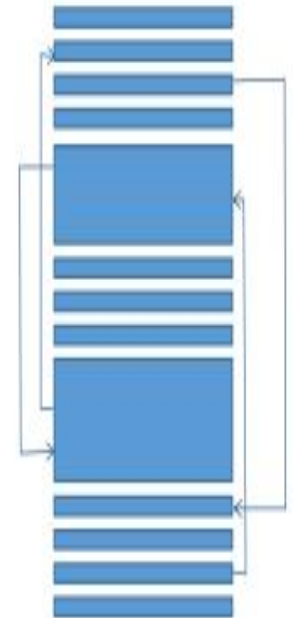
    start: //标号,是跟有冒号的标识符
    if (count>10)
        goto end;
    printf("%d ",count);
    count=count+1;
    goto start;

    end:
    printf("\n");
    system("pause");
    return 0;
}
```

运行效果:

1 2 3 4 5 6 7 8 9 10
请按任意键继续...

- 面条式的程序:
 - 文本空间中的程序:
 - 下锅前的面条
 - 运行中的执行序列:
 - 下锅后的面条
- 难以想象!
 - 文本空间和时间空间上巨大的gap





引言

建筑物需要有良好的结构来抵御外界风险。

同样，一个程序也应该具有良好的结构，易于理解和维护。

结构化程序设计方法(Structured programming method)

- 自顶向下 (Top-down)
- 逐步求精 (Progressive refinement)
- 模块化 (Module)
- 结构化编码 (Structured coding: three basic structure)

结构化编码：单入单出的三种控制结构

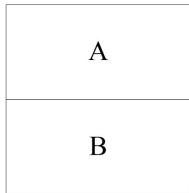


图4.15 顺序结构

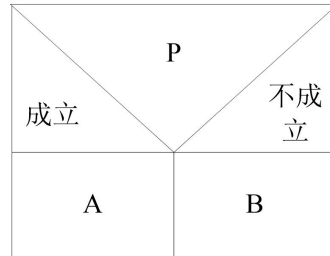


图4.16 选择结构

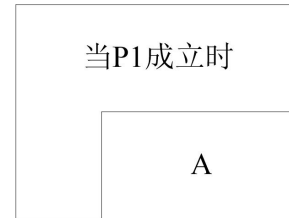


图4.17 当型循环结构

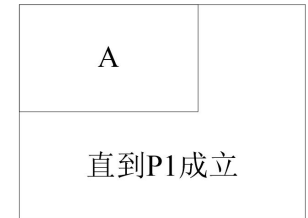


图4.18 直到型循环结构

三种控制结构的共同点

- 只有一个入口
- 只有一个出口
- 结构内的每一个部分都有机会被执行到
- 结构内不存在“死循环”（无终止的循环）

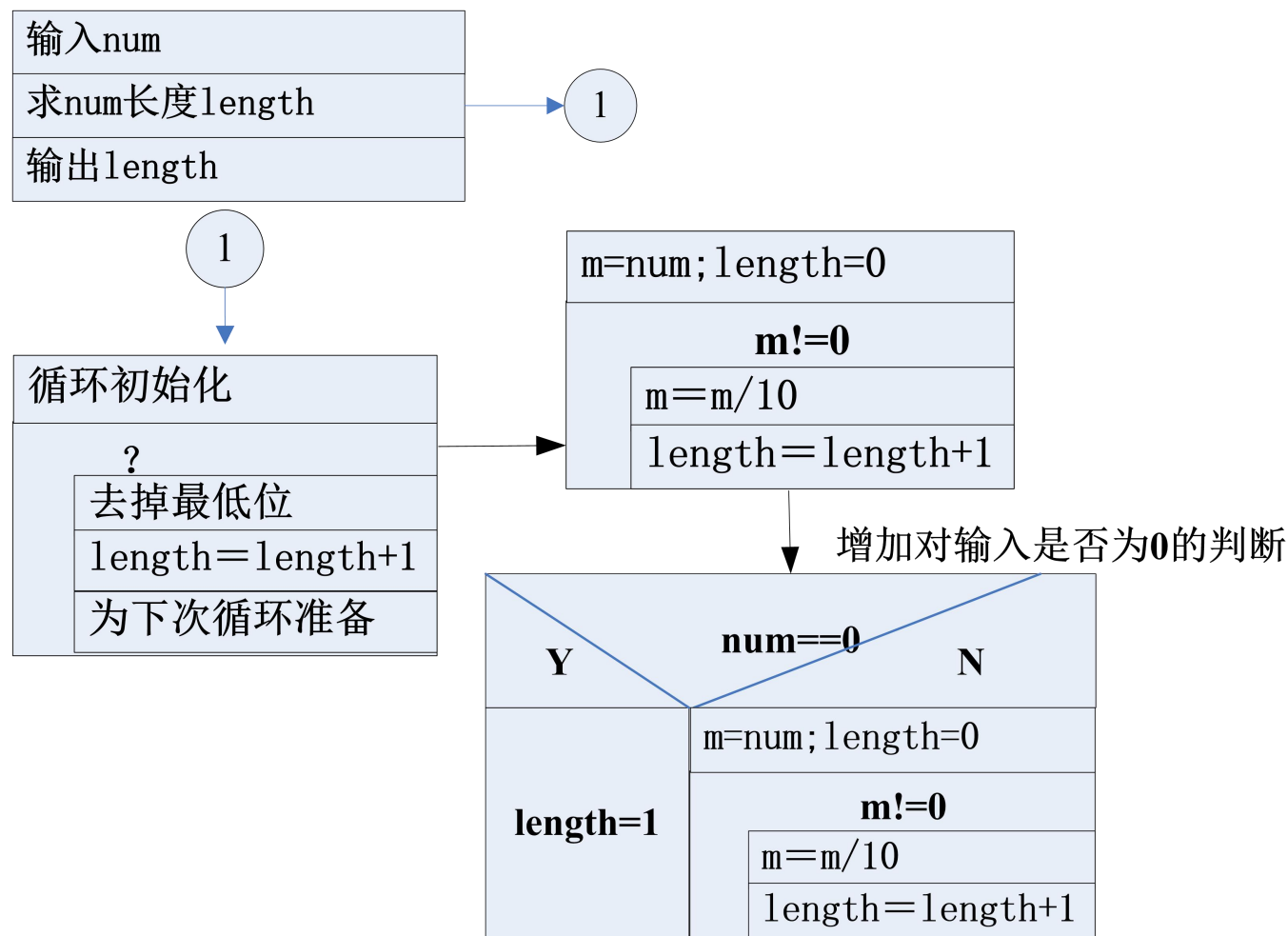
提倡 清晰的 结构

例4.14 求整数长度

- 练习3：设计算法，求一个非负整数的长度。
- 算法1：例如 $\text{num}=12345$ ，求 num 的长度
 - 第1步：去掉最低位， $\text{num}=1234$ ，长度 $\text{len}=1$
 - 第2步：去掉最低位， $\text{num}=123$ ，长度 $\text{len}=2$
 - 第3步：去掉最低位， $\text{num}=12$ ，长度 $\text{len}=3$
 - 第4步：去掉最低位， $\text{num}=1$ ，长度 $\text{len}=4$
 - 第5步：去掉最低位， $\text{num}=0$ ，长度 $\text{len}=5$

算法中每一步的核心操作是从 num 中去掉最低位，长度 len 加1

自顶向下、逐步求精：简化计算



按去低位迭代的求整数长度

```
#include<stdio.h>
int main(){
    int num,length,m;//输入的数,记录整数长
    printf("Enter a integer:");
    scanf("%d",&num);
    length=0;    m=num;
    //从最低位开始,依次从m中砍掉各位
    while (m != 0){
        m=m/10;//去掉m的最低位
        length++; /*从num中分解出一位数字,
    }
    printf("length of %d is %d\n",num,length);
}
```

```
int num,length,m;
```

```
printf("Enter a integer:");
```

```
scanf("%d",&num);
```

```
length=0;
```

```
m=num;
```

```
(m != 0)
```

```
m=m/10;
```

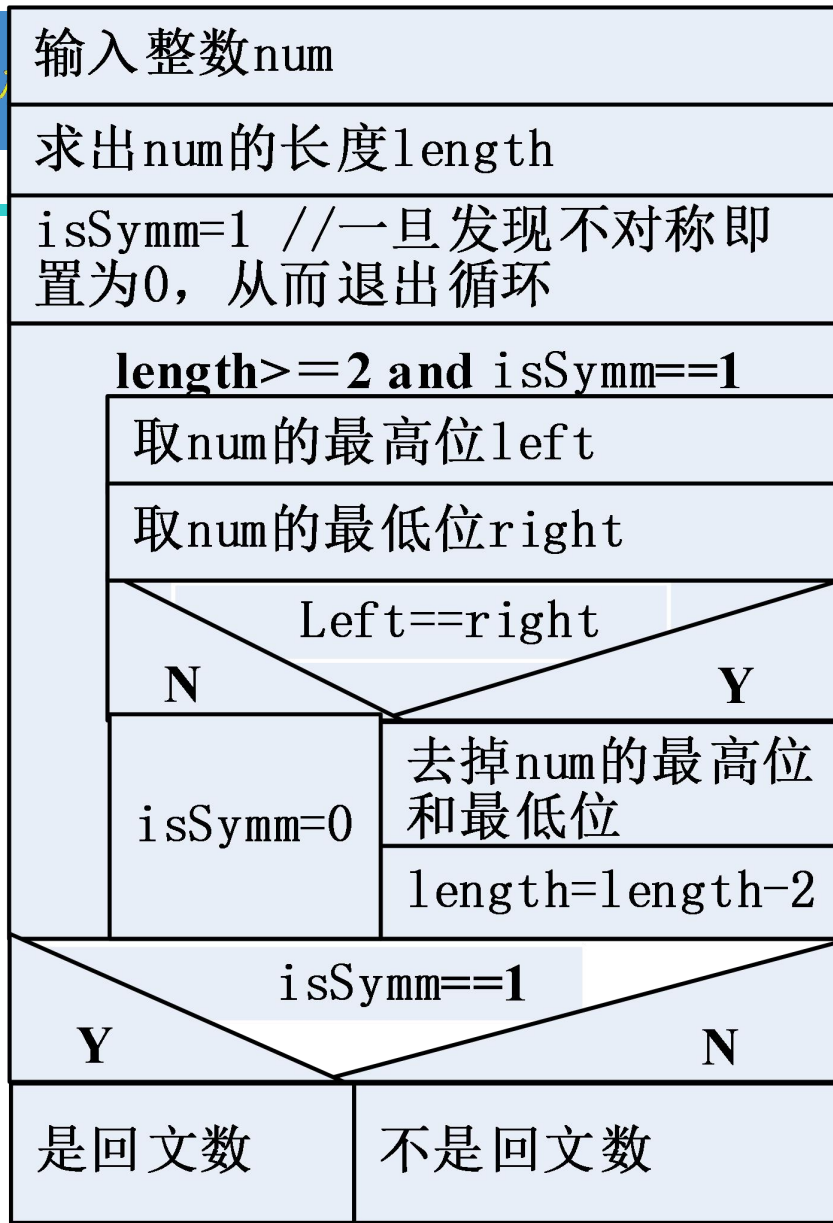
```
length++;
```

```
printf("length of %d is %d\n",num,length);
```

练习5：判断回文数

- 练习5：设计算法，判断一个正整数是否是回文数。回文数是指正读和反读都一样的数。如121、1221就是回文数。
- 算法1：回文数的第i位和倒数第i位肯定相等
- $f(n) = \{ \text{if } (n/10^{\text{len}-1}) == n \% 10$
 $\quad f((n \% 10^{\text{len}-1})/10), n \neq 0 \quad \}$
 - 比较第1位和倒数第1位
 - 比较第2位和倒数第2位
 - ...
- 算法2：反着读，假设读得的数为reverseNum，
判断num和reverseNum是否相等

判断回文数-算法1

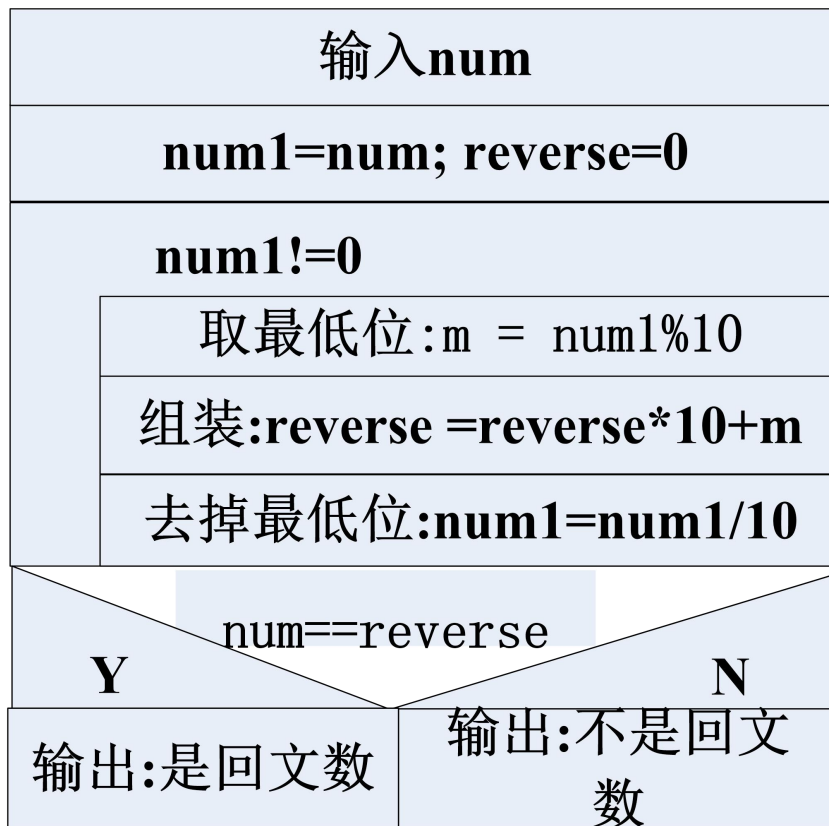


→ $left = num / 10^{\text{len}-1}$

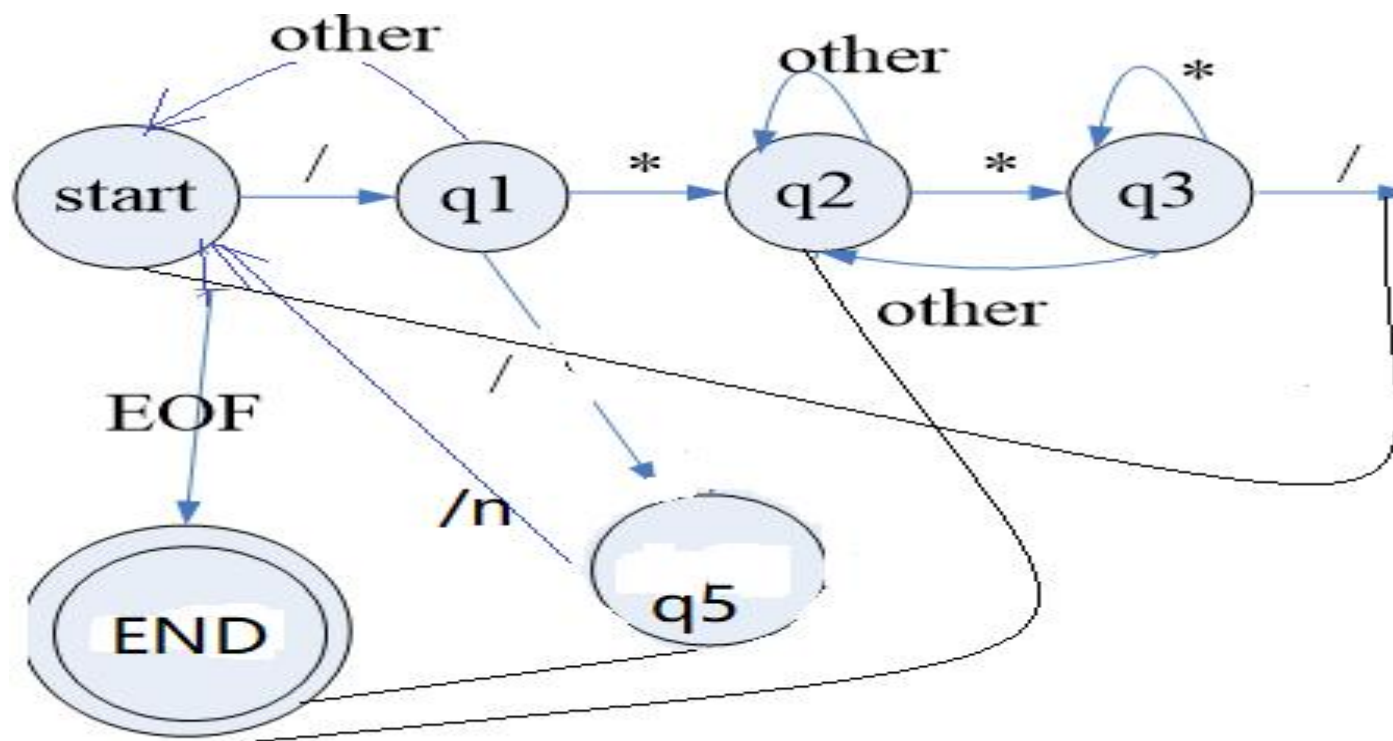
→ $right = num \% 10$

→ $num = \frac{num \% 10^{\text{len}-1}}{10}$

判断回文数-算法2



练习：读入多行字符串，将其中的行注释和段注释去掉并打印其余字符。





```
char ch;
```

```
int state;
```

```
state=START;
```

```
((ch=getchar())!='$')
```

(state)

case START:

(ch=='/')

state=Q1;

putchar(ch);

(ch=='*')

state=Q2;

(ch=='/')

state=Q4;

putchar('/');

putchar(ch);

state=START;

case Q1:

(ch=='*')

state=Q3;

case Q2:

(ch)

case '/':

state=START;

case '*':

do nothing

default:

state=Q2;

case Q3:

(ch=='\n')

putchar('\n');

state=START;

case Q4:

